

GPT-2 Arm-ו Retrieval ,HUB_DIM Collapse

סיכום והסבר ברור של הדיון: בחירת ממד ה-Hub, שחזור, Verification, R@1, SVD, AUC, anisotropy, hubness וקריסת זרוע GPT-2

GPT-2

Verification

Retrieval

Structural Similarity

Shared Hub

הרעיון המרכזי: HUB_DIM צריך להיות גדול מספיק כדי לשמר מידע, אך לא גדול עד כדי כך שהמיפוי אליו נהיה חלש סטטיסטית. במקביל, ביצועי retrieval יכולים לקרוס גם כאשר עדיין נשמר signal חלש יותר שנראה ב-Verification AUC.

תוכן עניינים

- 1. בחירת HUB_DIM על סמך מדידה
- 2. למה Hub קטן מדי מאבד מידע
- 3. למה Hub גדול מדי עלול להיות underpowered
- 4. Sweep ו-SVD slicing
- 5. R@1 לעומת Verification AUC
- 6. מהו GPT-2 arm collapse
- 7. הקשר בין collapse, anisotropy ו-hubness
- 8. למה AUC יכול לשרוד כאשר R@1 קורס
- 9. כיצד להבדיל intrinsic GPT-2 anisotropy מ-hub-induced collapse
- 10. סיכום קצר לבחינה / דו"ח

1. בחירת HUB_DIM על סמך מדידה

HUB_DIM הוא מספר הממדים של המרחב המשותף - ה-Hub - שאליו ממפים ייצוגים ממודלים או מרחבים שונים.



הבחירה ב-HUB_DIM היא trade-off בין שתי דרישות מנוגדות:

בחירה	יתרון	סיכון
Hub צר	קל יותר ללמוד mapping אמין	compression חזק מדי ואובדן מידע
Hub רחב	יכולת טובה יותר לשמר/לשחזר מידע	פחות דוגמאות לכל ממד; mapping יכול להפוך underpowered

לכן לא בוחרים את הערך לפי אינטואיציה בלבד. מבצעים **sweep** על מספר ערכים ומודדים reconstruction/retention, יחס rows-per-dimension ותוצאת transfer.

המסר של המשפט "the choice is made from evidence rather than argued":
לא אומרים בדיעבד "1024 נשמע סביר". מראים ניסויית איזה ערך מספק את האיזון הטוב ביותר.

2. למה Hub קטן מדי מאבד מידע

נניח שמרחב היעד הוא ברוחב 2048 ממדים, אך ה-Hub הוא רק 256 ממדים:

$$2048 \rightarrow 256 \rightarrow 2048$$

אם המבנה החשוב של היעד דורש יותר מ-256 כיוונים עצמאיים, ה-Hub הופך ל-**bottleneck**. הוא אינו יכול לשמר את כל המידע.

$$B \rightarrow H \rightarrow B_{\text{hat}}$$

If the hub is too narrow:

$$B_{\text{hat}} \neq B$$

זו המשמעות של:

“the hub cannot RECONSTRUCT a wide target”

כלומר, הבעיה אינה רק "ממד קטן" באופן מופשט; היא נמדדת בפועל באמצעות איכות reconstruction או retention.

3. למה Hub גדול מדי עלול להיות underpowered

ככל שמספר ממדי ה-Hub גדל, יש יותר דרגות חופש במיפוי שצריך ללמוד. מספר הדוגמאות, לעומת זאת, נשאר קבוע.

אם יש 10,000 rows:

$$\text{HUB_DIM} = 256$$

$$\text{rows per dimension} = 10000 / 256 \approx 39.1$$

$$\text{HUB_DIM} = 2048$$

$$\text{rows per dimension} = 10000 / 2048 \approx 4.88$$

בפרויקט שעליו דיברנו, הסף שנקבע הוא לפחות:

$$N / \text{HUB_DIM} \geq 5$$

כאשר היחס יורד מתחת לסף, המיפוי אל ה-Hub מקבל פחות תמיכה סטטיסטית לכל ממד. במילים פשוטות: מנסים לאמוד מרחב רחב מדי ביחס לכמות הנתונים.

the largest useful dimension היא ה-Hub הרצוי אינו בהכרח הרחב ביותר, אלא **Sweet spot**:
supported by the data

4. SVD slicing ו- Sweep

במקום לבחור ערך בודד, בודקים למשל:

256, 512, 768, 1024, 1536, 2048

ולכל רוחב מודדים לפחות:

1. **Reconstruction / retention** - כמה מהמידע המקורי נשמר.
2. **Rows per hub dimension** - האם יש מספיק נתונים ביחס לרוחב.
3. **Transfer performance** - האם ההתאמה בין המרחבים נשמרת לאחר המעבר דרך ה-Hub.

אם משתמשים ב-SVD:

$$X = U \Sigma V^T$$

מחשבים את ה-SVD פעם אחת, ואז משתמשים ב-prefix שונה של הרכיבים:

Hub 256 → first 256 singular components
Hub 512 → first 512 singular components
Hub 1024 → first 1024 singular components

לכן ניתן לבצע sweep על מספר רוחבים בעלות נוספת קטנה יחסית.

5. R@1 לעומת AUC Verification

R@1 - שאלת retrieval חזקה

עבור query מסוים A_i , יש הרבה candidates בצד השני:

$$B_1, B_2, \dots, B_n$$

וה-partner האמיתי הוא B_i . R@1 שואל:

$$\operatorname{argmax}_j \operatorname{sim}(A_i, B_j) = i ?$$

כלומר: האם הזוג הנכון הוא **מקום ראשון מכל המועמדים**?

Verification AUC - שאלה חלשה יותר

כאן שואלים האם score של true pair נוטה להיות גבוה מ-score של random pair:

$$\text{sim}(A_i, B_i) > \text{sim}(A_i, B_j), \quad j \neq i$$

או באופן הסתברותי:

$$P(\text{score_true} > \text{score_random})$$

AUC גבוה פירושו שיש יכולת טובה להפריד בין positive pair ל-negative/random pair, אך הוא אינו דורש שה-positive יהיה המועמד מספר 1 מתוך אלפים.

דוגמה: AUC גבוה אך R@1 נמוך

נניח שה-zוג האמיתי מקבל:

$$\text{true pair} = 0.80$$

ורוב הזוגות הלא נכונים מקבלים 0.30-0.60, אך קיימים שני negatives עם:

$$0.83, 0.82$$

ה-zוג האמיתי עדיין נראה הרבה יותר טוב מזוג אקראי טיפוסי, ולכן AUC Verification יכול להיות גבוה. אבל הוא אינו במקום הראשון, ולכן R@1 עבור אותה query נכשל.

מסקנה: Verification בודק אם נשאר correspondence signal. Retrieval, ובמיוחד R@1, בודק אם ה-signal חד מספיק כדי לזהות את ה-partner הנכון מבין הרבה מועמדים.

6. מהו GPT-2 arm collapse?

בהקשר של ניסוי shared-hub / structural similarity, הביטוי מתאר מצב שבו **הצד של GPT-2 מאבד discrimination** גיאומטרי שימושי.

Model A $\longrightarrow$ HUB $\longleftarrow$ GPT-2

במצב תקין, ייצוגי GPT-2 שונים אמורים לשמר הבדלים משמעותיים:

$g_1, g_2, g_3, \dots$ are meaningfully distinct

ב-collapse, הרבה ייצוגים נעשים דומים מדי מבחינה גיאומטרית:

$g_1 \approx g_2 \approx g_3 \approx \dots$

התוצאה: ה-score של ה-target הנכון אינו מובחן מספיק מה-scores של candidates שגויים.

למה cosine גבוה אינו בהכרח טוב?

לפני collapse אפשר לראות:

true similarity = 0.85
random similarity = 0.20

אחרי collapse:

true similarity = 0.92
random similarity = 0.88

למרות שה-cosine של הזוג הנכון עלה, ה-margin בין true ל-random כמעט נעלם. לכן הייצוג פחות שימושי ל-retrieval.

7. הקשר בין collapse, anisotropy ו-hubness

Anisotropy

במרחב isotropic, vectors מפוזרים בהרבה כיוונים. ב-anisotropy חזק, רבים מהם מצביעים לכיוון כללי דומה או נמצאים בתוך cone צר.

Isotropic: ↗ ← ↓ ↘ ↑
 Anisotropic: ↗ ↗ ↗ ↗ ↗

כאשר קיים common component חזק, cosine similarity יכול להיות גבוה גם לזוגות שאינם תואמים.

Hubness

Hubness הוא מצב אחר: candidates מסוימים מופיעים כ-nearest neighbor של מספר חריג של queries.

many queries → the same few candidate vectors

לכן:

מושב	מה הוא מתאר?
Anisotropy	הווקטורים מרוכזים מדי בכיוונים דומים.
Hubness	ווקטורים מסוימים נהפכים לשכנים של יותר מדי queries.
Arm collapse	התוצאה הפרקטית: הצד של GPT-2 אינו מספק מספיק discrimination ל-transfer/retrieval אמין.

8. למה AUC יכול לשרוד כאשר R@1 קורס?

נניח שאחרי transfer מתקבלים scores:

True pairs: 0.87, 0.89, 0.86, 0.90
 Random pairs: 0.75, 0.78, 0.80, 0.77

ברמת distribution, true pairs עדיין נוטים להיות גבוהים יותר, ולכן AUC נשאר טוב.

אבל retrieval מתבצע מול אלפים או עשרות אלפים של candidates. מספיק שמספר קטן של negatives יקבלו scores גבוהים במיוחד כדי לעקוף את הזוג הנכון:

wrong candidates: 0.91, 0.90, 0.89

לכן ייתכן:

Verification AUC = good
 R@1 = poor

הפרשנות: נשאר signal גם של correspondence, אך אבד חלק מהמידע המקומי/הפרטני הדרוש ל-exact matching.

9. Intrinsic GPT-2 anisotropy לעומת hub-induced collapse

חשוב להבדיל בין שתי תופעות:

א. אנאיזוטרופיה פנימית של GPT-2

הבעיה קיימת כבר בייצוג המקורי של GPT-2 לפני שמיפינו אותו ל-Hub. אם representations מקוריים כבר קרובים מאוד זה לזה, ה-Hub מתחיל מנקודת פתיחה קשה.

ב. Collapse שנגרם על ידי ה-Hub

כאן אובדן ההבחנה נוצר או מחמיר כתוצאה מ-projection, low-rank truncation, mapping לא יציב או bottleneck קטן מדי.

GPT-2: 768 dims → Hub: 64 dims → reconstructed GPT-2

אם הכיוונים שמבדילים בין examples נמצאים ברכיבים שנחתכו, אז לאחר reconstruction מתקבלים vectors דומים יותר:

original: g_1, g_2, g_3 are distinct
reconstructed: $\hat{g}_1 \approx \hat{g}_2 \approx \hat{g}_3$

שרשרת סיבתית אפשרית

GPT-2 anisotropy
↓
less effective variation
↓
restrictive hub / projection
↓
additional loss of discrimination
↓
GPT-2 arm collapse
↓
R@1 crashes
↓
AUC may still remain respectable

10. מה זה אומר לגבי Structural Similarity?

Collapse בצד GPT-2 אינו מוכיח בהכרח שאין shared structure בין המודלים.

ייתכנו לפחות שתי פרשנויות:

1. אין מספיק structural correspondence אמיתי.
2. יש correspondence, אך measurement pipeline איבד את היכולת לפתור אותו ברזולוציה הנדרשת.

אם AUC עדיין גבוה בזמן ש-R@1 קרס, האפשרות השנייה נהיית סבירה יותר: יש signal שנשמר, אבל אינו מספיק ל-identification חד-חד-ערכי.

ניסוח שימושי לדו"ח:

The GPT-2 arm may preserve coarse correspondence while losing pair-specific discriminability. A collapse in R@1 therefore does not by itself establish absence of shared structure; it may instead indicate that the mapping or hub geometry no longer resolves that structure at retrieval precision

11. סיכום קצר לבחינה / דו"ח

- HUB_DIM קטן מדי → reconstruction/retention → bottleneck נפגעים.
- HUB_DIM גדול מדי → פחות rows לכל mapping → dimension עלול להיות underpowered.
- בוחרים Hub באמצעות sweep מדיד, לא באמצעות נימוק אינטואיטיבי בלבד.
- SVD ניתן לחשב פעם אחת ואז לחתוך לרוחבים שונים.
- R@1 שואל האם ה-target האמיתי הוא מקום ראשון מתוך כל candidates.
- Verification AUC שואל האם true pairs מובחנים סטטיסטית מ-random pairs.
- לכן AUC יכול להישאר גבוה גם כאשר R@1 נמוך.
- GPT-2 arm collapse = אובדן discriminative geometry בצד GPT-2.
- Anisotropy = כיוונים דומים מדי; hubness = מסוימים נהפכים לשכנים של יותר מדי queries.
- Collapse אינו לבדו הוכחה להעדר structural similarity; הוא יכול להצביע על כשל רזולוציה של ה-pipeline.

המסמך נערך מתוך הדיון הנוכחי בצ'אט, בניסוח לימודי ומאורגן.

Update: G1/G2 Shared Hub, Head, Query Flow, and New-Model Integration

Added from the recent discussion. This section separates the Hub, encoder-to-Hub maps, the Hub-to-bge head, E1's direct DINOv2-to-bge experiment, query-time retrieval, and GPT-2's role.

Most important distinction: the 512-d object belongs to G1's shared Hub. It is not the same thing as E1's direct DINOv2-to-bge map. In G1 the path is encoder -> Hub(512) -> head -> bge(1024).

10. Predicted effective rank

Predicted effective rank means the number of directions expected to carry useful, independent signal, even when the nominal vector width is much larger.

nominal dimension = 768, but usable effective rank may be about 6.5

If most spectral mass sits in only a few singular directions, the space behaves as if it were much lower dimensional. This matters because a wide space can still have very poor cosine discrimination if nearly all vectors occupy the same narrow cone.

GPT-2 example from the project: raw GPT-2 target geometry had effective rank about 6.5/768 and mean pair-cosine about +0.999. That is why raw cosine can look almost perfect while retrieval remains poor.

11. What are the 8,533 "same items"?

G1 uses 9,533 aligned semantic items in total: **8,533 for fitting** and **1,000 held out for evaluation**.

"Same item" does **not** mean every model receives the same raw input. It means the rows refer to the same underlying COCO image-caption item:

Branch	Input for row i	Output width
DINOv2-small	image i (pixels)	768
DINOv2-base	image i (pixels)	1536
DINOv2-large	image i (pixels)	2048
bge-m3	caption(s) describing image i	1024
GPT-2	caption(s) describing image i	768

Example row:

Image: a cat sitting on a chair.

Text: "A cat is sitting on a chair."

The three DINO branches encode the pixels. bge-m3 and GPT-2 encode text describing the same referent.

12. How the 512-d Hub is created

For each of the 8,533 fitting items, take the five aligned representations and standardize each model space separately.

$$768 + 1536 + 2048 + 1024 + 768 = 6144$$

Then concatenate the five standardized representations row by row:

$$z_i = [x_{\text{small}} \mid x_{\text{base}} \mid x_{\text{large}} \mid x_{\text{bge}} \mid x_{\text{gpt2}}] \text{ in } \mathbb{R}^{6144}$$

Across all fitting rows:

$$Z \text{ in } \mathbb{R}^{(8533 \times 6144)}$$

Apply PCA/SVD, keep the top 512 components, then whiten them. The resulting training-coordinate matrix is:

$$H \text{ in } \mathbb{R}^{(8533 \times 512)}$$

What is stored? The project can cache the 8,533 training Hub vectors, but the important reusable objects are the Hub basis/normalization and the model-specific linear maps. For a new item, its 512-d Hub vector is computed on the fly.

13. One encoder-to-Hub map per model

Once H is defined, each encoder learns how to express its own vectors in the common 512-d coordinates:

$$X_k W_k \approx H$$

Map	Shape
W_small	768 x 512
W_base	1536 x 512
W_large	2048 x 512
W_bge	1024 x 512
W_gpt2	768 x 512

At runtime only one encoder is needed. Example:

```
new image -> DINOv2-base -> x_base(1536) -> W_base -> h(512)
```

The original 6144-d concatenation is a **Hub-construction operation**, not a production inference requirement.

14. How the caption-retrieval head is created

The head is a separate linear map:

$$W_{\text{head}} : R^{512} \rightarrow R^{1024}$$

It is fitted using only the DINOv2-small arm after that arm has already entered the Hub:

$$H_{\text{small}} W_{\text{head}} \approx T_{\text{bge}}$$

Here H_{small} contains 512-d Hub vectors produced from DINOv2-small, and T_{bge} contains the corresponding native bge-m3 caption targets.

Why this is a strong test: after fitting, W_{head} is frozen. The same head is then fed Hub vectors coming from DINOv2-base and DINOv2-large, which it never saw during head fitting. The reported transfer reaches 95.9% and 92.9% of the corresponding native-head performance.

15. What does t-hat mean?

The hat means **predicted**.

$$t_hat_i = h_i W_head$$

t_i would be the native bge-m3 embedding of the caption. t_hat_i is the image's predicted vector in bge-m3 space after going through the Hub and head.

image -> DINO -> Hub vector h_i -> W_head -> predicted bge vector t_hat_i

16. Query flow example: "cat sit on a chair"

Offline image indexing

image_j -> DINOv2-base -> W_base -> $h_j(512)$ -> W_head -> $t_hat_j(1024)$

Store each t_hat_j in the retrieval index.

Online text query

$q = \text{bge}(\text{"cat sit on a chair"})$ in $\mathbb{R}^{1024}$

Now both query and indexed images are in bge space:

$\text{score}_j = \text{cosine}(q, t_hat_j)$

Rank the gallery by score and return the top-k images.

The Hub is the interoperability layer; bge-m3 is the final retrieval space.

The normal text query does not need to pass through GPT-2 or through the Hub because bge already produces a native vector in the target retrieval space.

17. How GPT-2 can contribute

1. **During Hub construction:** GPT-2 caption representations are one of the five representation blocks used to define the joint PCA/whitened Hub basis.
2. **As its own Hub arm:** GPT-2 has a fitted map $W_gpt2 : 768 \rightarrow 512$, so GPT-2 text can in principle be placed into the Hub.
3. **Not required for normal bge retrieval:** for a text query, $\text{text} \rightarrow \text{bge}$ is the direct and appropriate path.

A possible experimental alternative is:

text -> GPT-2 -> W_gpt2 -> Hub -> W_head -> bge space

But the supplied project results do not establish this exact end-to-end path as a high-performing deployed query route. GPT-2's raw geometry is especially problematic because its effective rank is very low and cosine is nearly degenerate.

18. Adding a new encoder to the existing Hub

Suppose a new vision model outputs $d_{\text{new}} = 1280$ dimensions.

Preferred modular integration

1. Keep the existing 512-d Hub fixed.
2. Run the new model on aligned items for which the desired Hub coordinates are already known.
3. Fit only one new ridge map:

$$X_{\text{new}} W_{\text{new}} \approx H, W_{\text{new}} \text{ in } \mathbb{R}^{(1280 \times 512)}$$

Then use:

new model -> W_new -> existing Hub -> existing W_head -> bge space

Engineering advantage: adding a model can mean fitting one new model-to-Hub map rather than building pairwise adapters to every existing model.

If you rebuild the Hub

If you concatenate six spaces and rerun PCA, the Hub axes can change. Even if the width remains 512, the old head may no longer interpret the new coordinates correctly. A rebuilt Hub can therefore require refitting or realigning existing maps and heads.

This frozen-Hub sixth-model integration is the natural architectural extension of G1, but the supplied report does not claim it has already been demonstrated for an arbitrary unseen sixth encoder.

19. Do not confuse E1 with G1

	E1 direct cross-modal map	G1 shared-Hub system
Core path	DINOv2(768) -> W -> bge(1024)	encoder -> Hub(512) -> head -> bge(1024)
Training data	8,000 COCO train2017 image-caption pairs	9,533 aligned items total; 8,533 fit + 1,000 eval
Fit/eval	7,000 fit + 1,000 held out	8,533 fit + 1,000 held out
Caption treatment	5 human captions encoded and averaged for one target	Text branches participate in the joint Hub and task-head evaluation
512-d object?	No shared 512-d Hub in the direct E1 map	Yes - central shared Hub

E1's five-caption averaging improved held-out R-squared from 0.278 to 0.511. The direct fit used $7,000/768 = 9.1$ rows per input dimension, with 1,000 rows held out. The matched ceiling must be measured on those same 1,000 rows, in the same image-to-text direction and with the same averaging protocol; under the matched protocol, the reported result is 40.9% of ceiling rather than the earlier inflated 55.2%.

20. One-line mental model

G1: build a common 512-d language from five representation spaces; teach each encoder one linear map into that language; train one task head on one encoder's Hub vectors; test whether the same head works when another encoder speaks the same Hub language.

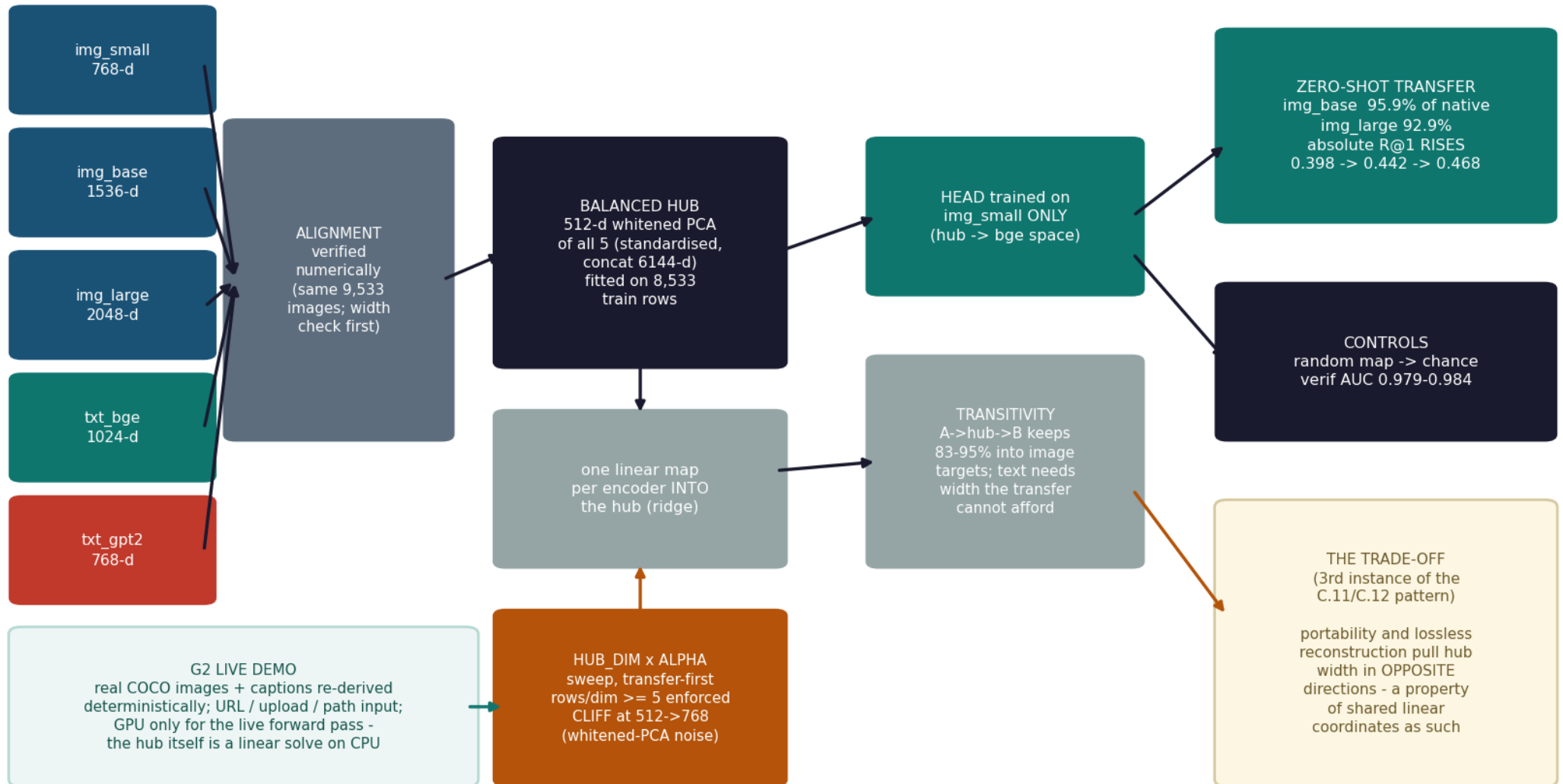
encoder -> W_encoder -> HUB_512 -> W_head -> BGE_1024 -> cosine retrieval

21. Visual appendix

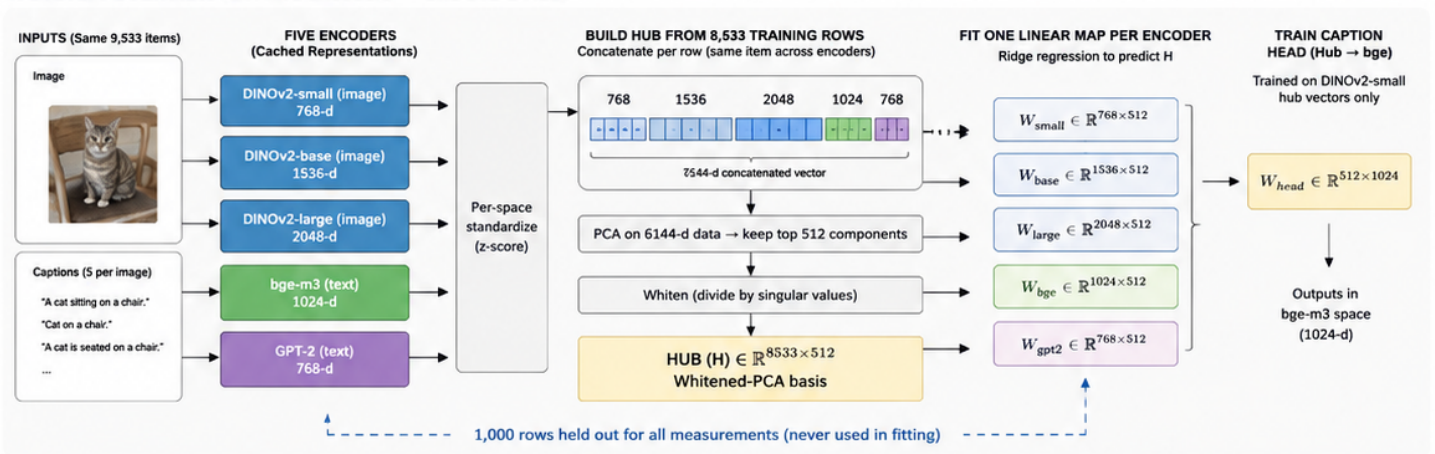
The following pages preserve the visuals created during the discussion. The latest direct-map visual is included in two forms: the original full poster and an enlarged split reading version.

1. Original supplied G1/G2 five-encoder flow.
2. Earlier full-system Hub infographic.
3. Higher-resolution Hub infographic with legend.
4. Simplified per-model input, query-flow, and GPT-2 contribution infographic.
5. Latest E1 direct DINOv2-to-bge infographic - full version.
6. Latest E1 infographic - enlarged top and bottom reading views.

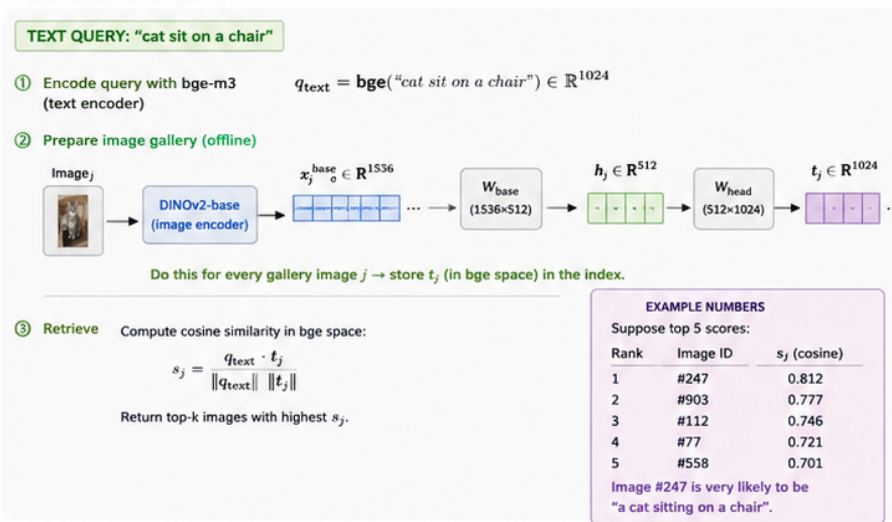
G1/G2 — five encoders, one 512-d hub: the shared space built, measured, and demonstrated live



1. SYSTEM OVERVIEW (G1: Five Encoders → One 512-d Hub)



2. QUERY FLOW EXAMPLE



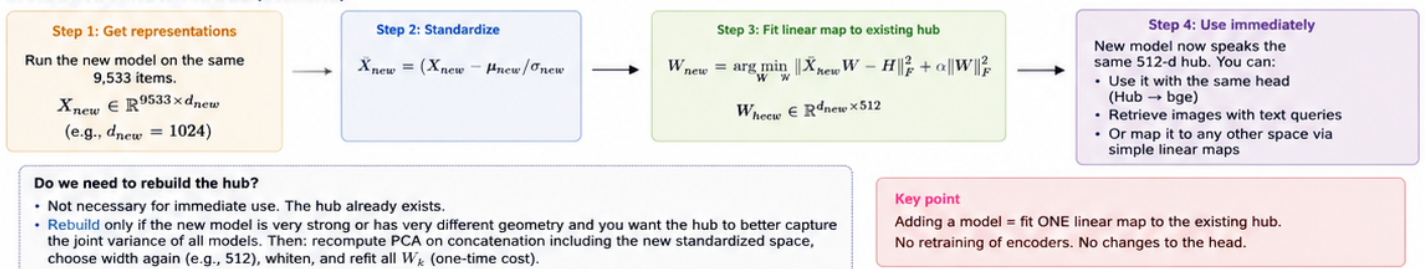
4. DETAILED DIMENSIONS

Component	Dim In	Dim Out	Shape
DINOv2-small	768	512	$W_{small} ∈ ℝ^{768 × 512}$
DINOv2-base	1536	512	$W_{base} ∈ ℝ^{1536 × 512}$
DINOv2-large	2048	512	$W_{large} ∈ ℝ^{2048 × 512}$
bge-m3	1024	512	$W_{bge} ∈ ℝ^{1024 × 512}$
GPT-2	768	512	$W_{gpt2} ∈ ℝ^{768 × 512}$
Hub → bge head	512	1024	$W_{head} ∈ ℝ^{512 × 1024}$

WHY IT WORKS

- Hub is learned from all five spaces together (whitened PCA).
- Each encoder learns how to express itself in that common coordinate system (linear maps).
- Head learned on one encoder's hub vectors can be reused on other encoders (zero-shot component transfer).
- Retrieval happens in bge space, where text queries live natively.

5. ADDING A NEW MODEL (Scenario)



3. MATH SUMMARY

Hub construction

- Standardize each space: $\tilde{X}_k = \frac{X_k - \mu_k}{\sigma_k}$
- Concatenate per row:
 $Z = [\tilde{X}_{small} | \tilde{X}_{base} | \tilde{X}_{large} | \tilde{X}_{bge} | \tilde{X}_{gpt2}] ∈ ℝ^{51333 × 6144}$
- PCA (SVD): $Z = U \Sigma V^T$
- Keep top 512: $V_{512} ∈ ℝ^{6144 × 512}$
- Whiten: $H = Z V_{512} \Sigma_{512}^{-1} ∈ ℝ^{8533 × 512}$

Encoder → Hub maps (ridge regression)

For encoder k with matrix $X_k ∈ ℝ^{8533 × d_k}$:

$$W_k = \arg \min_W \|X_k W - H\|_F^2 + \alpha \|W\|_F^2$$

Closed form: $W_k = (X_k^T X_k + \alpha I)^{-1} X_k^T H ∈ ℝ^{d_k × 512}$

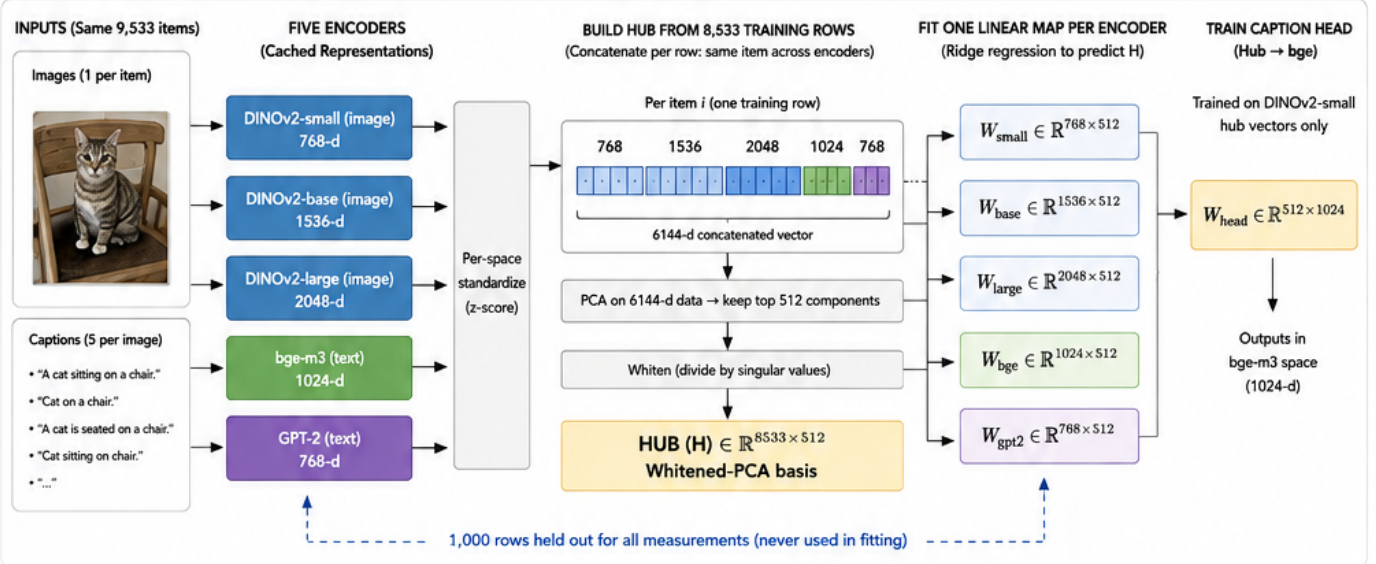
Caption head (Hub → bge)

Using only small encoder hub vectors $H_{small} ∈ ℝ^{8533 × 512}$ and bge targets $T_{bge} ∈ ℝ^{8533 × 1024}$:

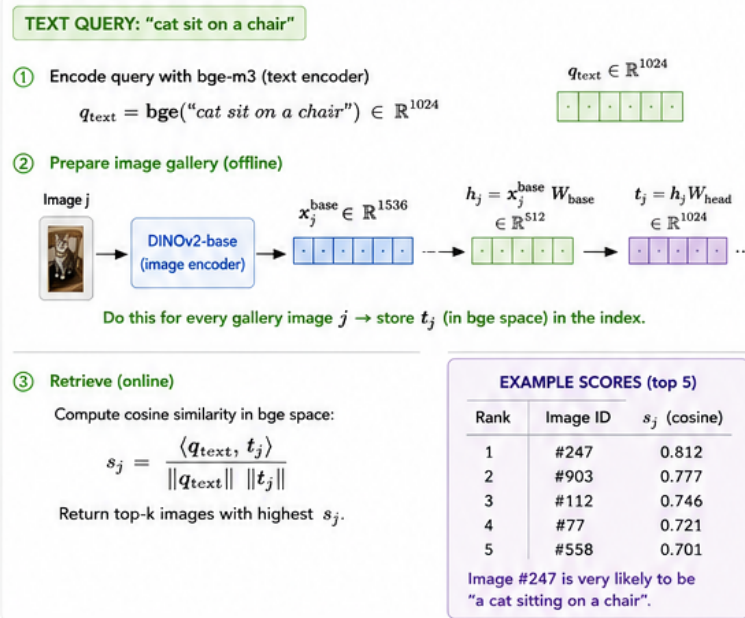
$$W_{head} = \arg \min_W \|H_{small} W - T_{bge}\|_F^2 + \lambda \|W\|_F^2$$

$$W_{head} = (H_{small}^T H_{small} + \lambda I)^{-1} H_{small}^T T_{bge} ∈ ℝ^{512 × 1024}$$

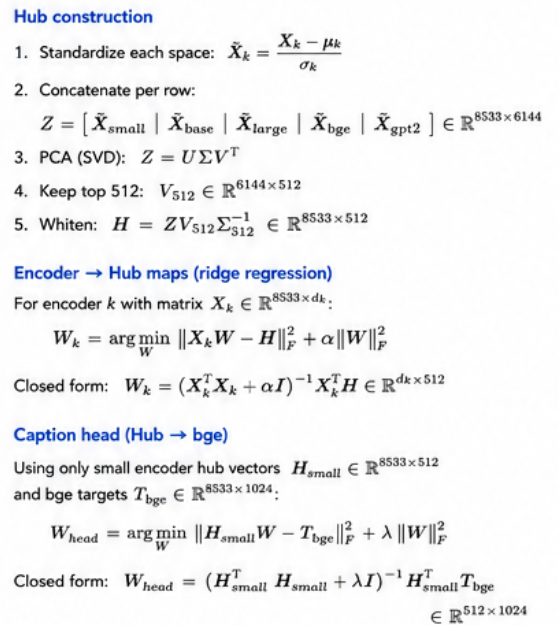
1. SYSTEM OVERVIEW (G1: Five Encoders → One 512-d Hub)



2. QUERY FLOW EXAMPLE



3. MATH SUMMARY



4. DETAILS

DIMENSIONS TABLE			
Component	Dim In	Dim Out	Matrix Shape
DINOv2-small	768	512	$W_{\text{small}} \in \mathbb{R}^{768 \times 512}$
DINOv2-base	1536	512	$W_{\text{base}} \in \mathbb{R}^{1536 \times 512}$
DINOv2-large	2048	512	$W_{\text{large}} \in \mathbb{R}^{2048 \times 512}$
bge-m3	1024	512	$W_{\text{bge}} \in \mathbb{R}^{1024 \times 512}$
GPT-2	768	512	$W_{\text{gpt2}} \in \mathbb{R}^{768 \times 512}$
Hub → bge head	512	1024	$W_{\text{head}} \in \mathbb{R}^{512 \times 1024}$

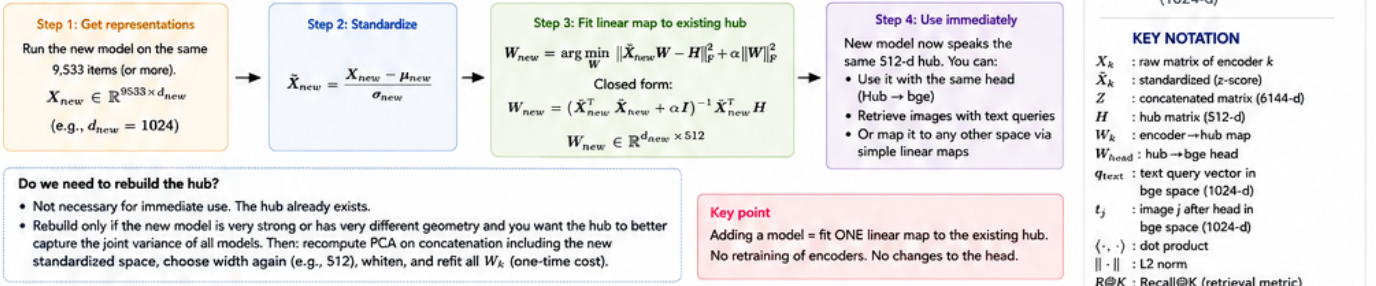
WHY IT WORKS

- ✓ Hub is learned from all five spaces together (whitened PCA).
- ✓ Each encoder learns how to express itself in that common coordinate system (linear maps).
- ✓ Head learned on one encoder's hub vectors can be reused on other encoders (zero-shot component transfer).
- ✓ Retrieval happens in bge space, where text queries live natively.

LEGEND

- Image encoders (DINOv2 family)
- Text encoder (contrastive)
- Text encoder (language model)
- Learned objects (Hub, Head)
- Preprocessing / operations
- Hub space (512-d)
- bge-m3 space (1024-d)

5. ADDING A NEW MODEL (Scenario)



IMPORTANT TAKEAWAYS



One hub for many encoders:
All models map linearly into the same 512-d space.



Component transfer:
A caption head trained on one encoder works on others it never saw (zero-shot).



Retrieval in text space:
Everything is converted to bge-m3 space (1024-d) for searching with text queries.

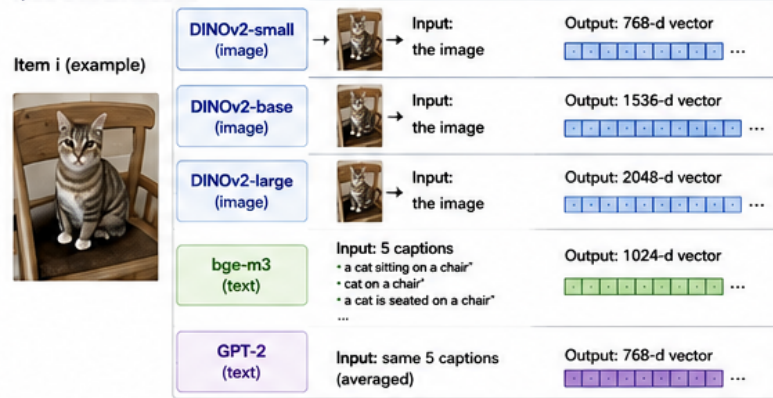


Extensible by design:
New models plug in by fitting one linear map—no retraining, no changes to the head.

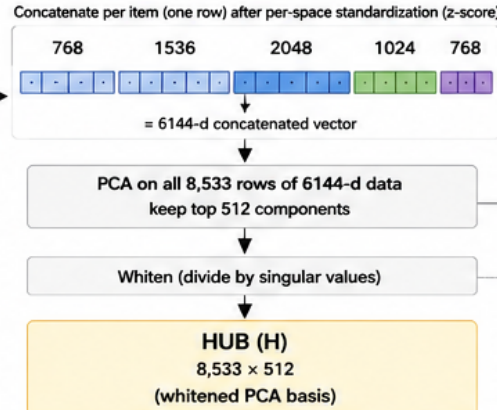
PART A. SYSTEM OVERVIEW (How the hub and maps are built)

We start with the SAME 8,533 training items (images with captions).

1) INPUT PER MODEL



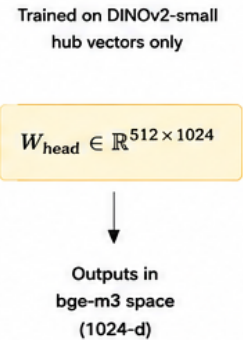
2) BUILD THE HUB (from all models together)



3) FIT ONE LINEAR MAP PER MODEL (Ridge regression to predict H)



4) TRAIN CAPTION HEAD (Hub → bge)



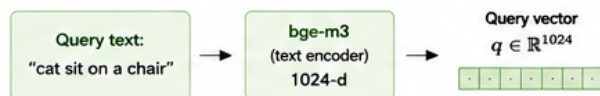
1,000 rows held out for all measurements (never used in fitting)

PART B. QUERY FLOW EXAMPLE (Find images for: "cat sit on a chair")

1) Prepare image gallery (offline once)



2) Process text query (online)



3) Retrieve

Compute cosine similarity in bge space

$$s_j = \frac{\langle q, \hat{t}_j \rangle}{\|q\| \|\hat{t}_j\|}$$

Return top-k images with highest s_j

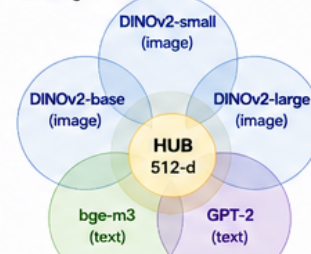
Example similarity scores (top 5)



PART C. CONTRIBUTION OF GPT-2 (How and if it helps)

During training (hub construction)

GPT-2 text representations are one of the five spaces used to build the hub. They influence the hub basis so that it captures directions useful for both images and language, even from a language model that was never trained with images.



At query time (does GPT-2 run?)



NORMAL PATH (recommended)

Use bge-m3 to encode the query text. (Faster and much better retrieval quality.)

Text (query) → bge-m3 → compare with image vectors (in bge space).



ALTERNATIVE PATH (possible but weaker)

You could use GPT-2 to encode the query and map it to hub:

Text → GPT-2 → W_{gpt2} → Hub → W_{head} → bge space → compare

This works, but typically lower accuracy than using bge-m3.

So GPT-2's main contribution is in building a better shared hub, not in being the best encoder for queries.

LEGEND

- Image encoder (DINOv2 family)
- Text encoder (contrastive: bge-m3)
- Text encoder (language model: GPT-2)
- Learned component (Hub / Head)
- Operations / processing
- bge-m3 space (1024-d)
- Hub space (512-d)

KEY DIMENSIONS

DINOv2-small	768 → 512
DINOv2-base	1536 → 512
DINOv2-large	2048 → 512
bge-m3	1024 → 512
GPT-2	768 → 512
Hub → bge head	512 → 1024

KEY TAKEAWAYS

- ★ One shared 512-d hub is learned from all five models together (8,533 training items).
- ★ Each model gets one linear map to the hub (ridge regression).
- ★ A caption head maps hub vectors to bge-m3 space (1024-d).
- ★ For text queries, use bge-m3 (best). GPT-2 can be used alternatively via Hub → Head, but is weaker.
- ★ The head trained on DINOv2-small transfers to unseen encoders (base/large) at ~93–96% of native performance.
- ★ 1,000 items are held out for all measurements (never used in fitting).

1. THE DATA (SUPERVISED BY HUMAN ANNOTATION)

IMAGE-CAPTION PAIRS (COCO)

Row i is an image **AND** the captions describing it.
Different objects sharing a referent.

Row i example
(one of 8,000)



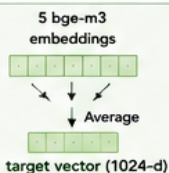
5 human captions
(COCO gives five per image)

- 1 "A cat sitting on a chair.
- 2 "Cat on chair.
- 3 "A cat is seated on a chair.
- 4 "Cat sitting on the chair.
- 5 "A cat perched on a wooden chair."
- ...

5 CAPTIONS AVERAGED

Each caption is encoded with bge-m3.
The five embeddings are averaged into one target vector.

→ Makes the target "what this image is about" rather than "what one annotator wrote".

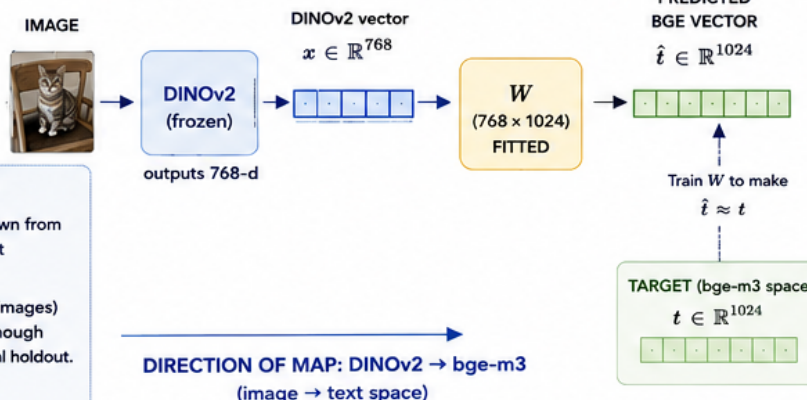


8k COCO train2017

- 8,000 such pairs drawn from COCO's training split (118k images).
- Not val2017 (5,000 images) because we need enough rows left after the eval holdout.

2. OVERALL FLOW (DINOv2 → BGE-M3)

Images go into DINOv2, one matrix W converts the resulting vectors into bge-m3's text space.
Both models are frozen; W is the only thing fitted.



3. HOW W IS FITTED (RIDGE REGRESSION)

Fit 7,000 (9.1 rows/dim)

- 7,000 rows train the ridge.
- DINOv2 outputs 768 numbers.
 $\frac{7,000}{768} 768 = 9.1$ rows per input dimension.
- Project rule: ≥ 5 required, 5–10 working band.
- Experiment A successful run sat at 9.8.
→ 9.1 is comfortably powered.
- Checked BEFORE results are read, so a weak outcome can't be blamed on starvation.

Ridge regression:

$$W = \arg \min_W \|XW - T\|_F^2 + \alpha \|W\|_F^2$$

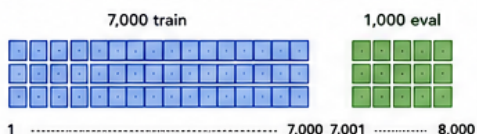
$X \in \mathbb{R}^{n \times 768}$ (DINOv2 vectors)

$T \in \mathbb{R}^{n \times 1024}$ (bge-m3 targets)

$n = 7,000$ (training rows)

4. TRAIN / EVAL SPLIT

Total: 8,000 pairs (COCO train2017)



TRAIN (7,000)

Used to fit W .
 W only sees these rows.

EVAL (1,000)

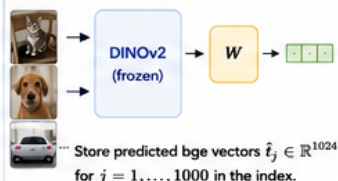
Held out completely.
Never used in fitting.

W is then frozen and used for evaluation only.

5. QUERY / RETRIEVAL FLOW (IMAGE → TEXT)

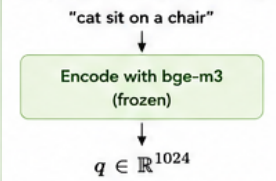
1) Encode eval gallery (offline)

Each of the 1,000 eval images:



2) Process caption query (online)

Query caption (averaged like training):



3) Retrieve (image → text)

For $j = 1 \dots 1000$, compute cosine similarity in bge space:

$$s_j = \frac{\langle q, \hat{t}_j \rangle}{\|q\| \|\hat{t}_j\|}$$

Return top- k images with highest s_j .

Example top-5 scores ($k=5$)



Each caption-vector queries a gallery of 1,000 candidates → chance = $1 / 1000 = 0.001$.

6. CEILING MEASUREMENT (CRITICAL FOR % SCORE)

Ceiling measured on the SAME 1,000 rows

- This is the clause that took three attempts today.
- The result is reported as a % of what a JOINTLY TRAINED model achieves, because captions are ambiguous and no independent pair can beat a model built for the task.
- Ceiling must be measured on the SAME rows, in the same direction (image→text), with the same caption treatment (averaged). When it wasn't, the score read 55.2%; matched properly, 40.9%.

Example (on the same 1,000 eval rows)

System	R@1 (image→text)	
Our linear map (DINOv2 → bge-m3)	0.409	(40.9% of ceiling)
Jointly trained model (ceiling)	1.000	(100%)
Chance (1/1000)	0.001	(0.1%)

★ **THE WHOLE LINE IN ONE SENTENCE:** 8,000 human-paired image-caption rows, 7,000 to fit one matrix at comfortable statistical power, 1,000 held out, and the benchmark computed on those same 1,000 rows under identical conditions so the percentage means.

LEGEND

- Image encoder (DINOv2)
- Text encoder (GPT-2)
- Learned component (Matrix W or Head)
- Text encoder (bge-m3)
- Operations / processing

- $x \in \mathbb{R}^{768}$ DINOv2 vector (image)
- $t \in \mathbb{R}^{1024}$ bge-m3 target vector (text)
- $\hat{t} \in \mathbb{R}^{1024}$ Predicted bge vector
- $W \in \mathbb{R}^{768 \times 1024}$ Fitted linear map (only trained thing)
- $q \in \mathbb{R}^{1024}$ Query vector (text)
- s_j Cosine similarity score
- R@K Recall@K

1. THE DATA (SUPERVISED BY HUMAN ANNOTATION)

IMAGE-CAPTION PAIRS (COCO)

Row i is an image **AND** the captions describing it.
Different objects sharing a referent.

Row i example
(one of 8,000)

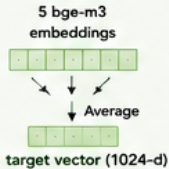


- 5 human captions
(COCO gives five per image)
- 1 "A cat sitting on a chair.
 - 2 "Cat on chair.
 - 3 "A cat is seated on a chair.
 - 4 "Cat sitting on the chair.
 - 5 "A cat perched on a wooden chair."
- ...

5 CAPTIONS AVERAGED

Each caption is encoded with bge-m3. The five embeddings are averaged into one target vector.

→ Makes the target "what this image is about" rather than "what one annotator wrote".



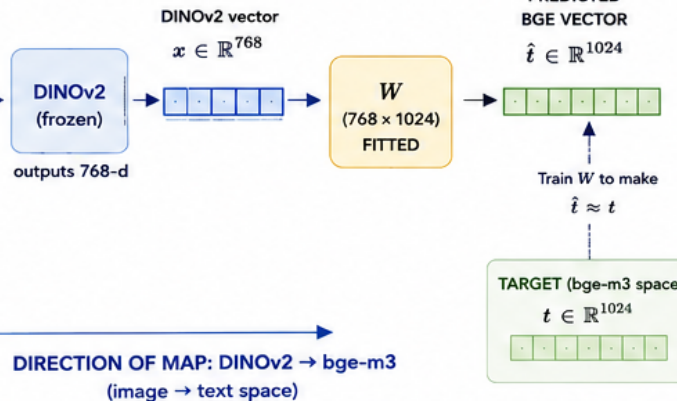
8k COCO train2017

- 8,000 such pairs drawn from COCO's training split (118k images).

- Not val2017 (5,000 images) because we need enough rows left after the eval holdout.

2. OVERALL FLOW (DINOv2 → BGE-M3)

Images go into DINOv2, one matrix W converts the resulting vectors into bge-m3's text space. Both models are frozen; W is the only thing fitted.



3. HOW W IS FITTED (RIDGE REGRESSION)

Fit 7,000 (9.1 rows/dim)

- 7,000 rows train the ridge.
- DINOv2 outputs 768 numbers.
- $\frac{7,000}{768} 768 = 9.1$ rows per input dimension.
- Project rule: ≥ 5 required, 5–10 working band.
- Experiment A successful run sat at 9.8. → 9.1 is comfortably powered.
- Checked BEFORE results are read, so a weak outcome can't be blamed on starvation.

Ridge regression:

$$W = \arg \min_W \|XW - T\|_F^2 + \alpha \|W\|_F^2$$

$X \in \mathbb{R}^{n \times 768}$ (DINOv2 vectors)

$T \in \mathbb{R}^{n \times 1024}$ (bge-m3 targets)

$n = 7,000$ (training rows)

4. TRAIN / EVAL SPLIT

Total: 8,000 pairs (COCO train2017)

7,000 train



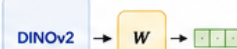
1,000 eval



5. QUERY / RETRIEVAL FLOW (IMAGE → TEXT)

1) Encode eval gallery (offline)

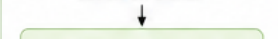
Each of the 1,000 eval images:



2) Process caption query (online)

Query caption (averaged like training):

"cat sit on a chair"



3) Retrieve (image → text)

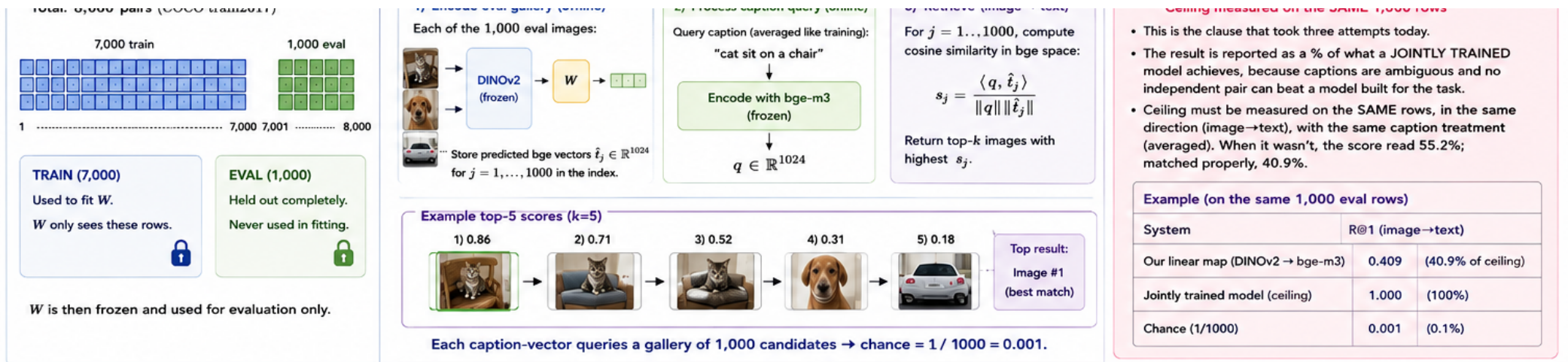
For $j = 1 \dots 1000$, compute cosine similarity in bge space:

$$\langle q, \hat{t}_j \rangle$$

6. CEILING MEASUREMENT (CRITICAL FOR % SCORE)

Ceiling measured on the SAME 1,000 rows

- This is the clause that took three attempts today.
- The result is reported as a % of what a JOINTLY TRAINED model achieves, because captions are ambiguous and no independent pair can beat a model built for the task.



★ **THE WHOLE LINE IN ONE SENTENCE:** 8,000 human-paired image-caption rows, 7,000 to fit one matrix at comfortable statistical power, 1,000 held out, and the benchmark computed on those same 1,000 rows under identical conditions so the percentage means.

LEGEND			
Image encoder (DINOv2)	Text encoder (GPT-2)	Learned component (Matrix W or Head)	
Text encoder (bge-m3)	Operations / processing		
		$x \in \mathbb{R}^{768}$ DINOv2 vector (image) $t \in \mathbb{R}^{1024}$ bge-m3 target vector (text) $\hat{t} \in \mathbb{R}^{1024}$ Predicted bge vector	$W \in \mathbb{R}^{768 \times 1024}$ Fitted linear map (only trained thing) $q \in \mathbb{R}^{1024}$ Query vector (text) s_j Cosine similarity score $R@K$ Recall@K